

A dificuldade de reprodução de ambientes computacionais em eScience: Uma análise comparativa entre as abordagens do Docker e do ReproZip.

1. Introdução e Motivação

Atualmente, a área de eScience (pesquisa voltada à análise de grandes volumes de dados) enfrenta um desafio crítico: garantir que o experimento de um pesquisador possa ser executado novamente em outra máquina, anos após a sua criação, tornou-se uma tarefa complexa. Essa dificuldade decorre da natureza dos fluxos de dados (pipelines), que envolvem múltiplas ferramentas, versões específicas de bibliotecas e configurações estritas de sistema operacional.

Ao buscar soluções para esse cenário, o Docker consolidou-se rapidamente como o padrão da indústria. Como Boettiger (2015) destacou no início da adoção dessa tecnologia pela comunidade acadêmica, a ferramenta trazia a promessa de "empacotar o ambiente computacional exato para que os resultados pudessem ser reproduzidos por qualquer pessoa".

Na prática, entretanto, estudos empíricos demonstram que a realidade é diferente. A elaboração de um arquivo Dockerfile exige a fixação manual de versões e o gerenciamento de dependências externas (como comandos `apt-get update`). Devido a esses fatores, pesquisadores como Malka et al. (2026), após avaliarem mais de 5.000 repositórios, concluíram de forma categórica que "o Docker não oferece garantia de reprodutibilidade". Os autores demonstraram que uma parcela significativa das imagens falha ao ser compilada após alguns anos, geralmente por conta de links inativos ou pacotes desatualizados, exigindo constante intervenção manual.

Como alternativa a essa limitação, o ReproZip apresenta uma abordagem fundamentalmente distinta. Segundo os criadores da ferramenta, Chirigati et al. (2016), a estratégia mais eficaz para preservar a pesquisa é "evitar que os cientistas precisem descobrir e instalar dependências manualmente". Em vez de exigir a configuração declarativa de um ambiente a partir do zero, o ReproZip opera monitorando a execução real do experimento. A ferramenta rastreia as chamadas de sistema (syscalls) e captura, de forma automática, apenas os arquivos, bibliotecas e variáveis de ambiente efetivamente utilizados, consolidando-os em um pacote autossuficiente (.rpz).

Diante deste contexto, o presente trabalho propõe a realização de um experimento prático e comparativo. O objetivo é avaliar o Docker e o ReproZip para compreender, por meio de dados empíricos, as limitações da abordagem de contêineres na preservação de um pipeline de dados a longo prazo, além de demonstrar os benefícios tangíveis da automação de proveniência oferecida pelo ReproZip.

2. Objetivos

Objetivo Geral:

Realizar uma análise empírica e comparativa das capacidades, dificuldades de implementação e desempenho do ReproZip em relação ao Docker no suporte à reprodutibilidade de experimentos baseados em pipelines de dados.

Objetivos Específicos:

1. Desenvolver um experimento-modelo representativo de um pipeline de dados em eScience.
2. Mensurar e comparar o esforço de configuração e a curva de aprendizado exigidos por ambas as ferramentas.
3. Avaliar métricas de desempenho, incluindo tempo de empacotamento, tamanho do artefato gerado e *overhead* no tempo de execução.
4. A resiliência dos artefatos para preservação a longo prazo.

3. Metodologia e Desenho Experimental

A pesquisa caracteriza-se como um estudo comparativo de natureza experimental. Para avaliar as ferramentas de forma justa e quantitativa, o estudo será conduzido em três frentes metodológicas:

3.1. O Experimento-Modelo (Pipeline de Dados)

Para simular o cenário real de um pesquisador lidando com o fluxo de entrada e saída (I/O) e com o gerenciamento de dependências complexas, será implementado um pipeline de dados em linguagem Python. O fluxo consistirá em três etapas principais:

- **Extração:** Leitura de um conjunto de dados estruturado (arquivo `.csv`) utilizando a biblioteca `pandas`.
- **Transformação e Análise:** Limpeza de dados e aplicação de processamento estatístico ou de aprendizado de máquina em nível básico, utilizando a biblioteca `scikit-learn`.
- **Geração de Artefato (Output):** Geração de uma visualização gráfica dos resultados (arquivo `.png`) por meio das bibliotecas `matplotlib` ou `seaborn`.

3.2. Métricas de Avaliação

O pipeline descrito será submetido a ambas as infraestruturas (Docker e ReproZip). Os resultados serão tabulados com base nas seguintes dimensões de análise:

- **Esforço de Configuração e Setup:** Avaliação qualitativa e quantitativa (tempo gasto) na elaboração da "receita" do ambiente (`Dockerfile`) em contraposição à execução dinâmica de rastreamento (`reprozip trace`).
- **Desempenho de Empacotamento:** Medição do tempo de geração do ambiente (`docker build` vs. `reprozip pack`) e comparação do tamanho final, em Megabytes (MB), da imagem gerada e do pacote empacotado.

- **Tempo de Execução (Overhead):** O pipeline será executado em três cenários para extração de médias de tempo de processamento:
 1. Nativamente na máquina hospedeira.
 2. Isolado via contêiner (**docker run**).
 3. Executado através da ferramenta de descompactamento e isolamento do ReproZip (**reprounzip**).

Todos testes no sistema Ubuntu 22 LTS.

3.3. Transparência e Repositório

Todo o código-fonte do pipeline, os conjuntos de dados de teste, os arquivos de configuração (Dockerfiles), os pacotes gerados (.rpz) e as planilhas de resultados serão versionados e disponibilizados em um repositório público (GitHub/GitLab). O repositório conterá um documento de instruções rigoroso para garantir a reprodutibilidade do próprio estudo.

4. Cronograma de Execução

As atividades necessárias para a conclusão do trabalho foram estruturadas para o período compreendido entre 29 de abril e a data de entrega estipulada.

Período	Etapas da Pesquisa	Atividades Previstas
Etapas 1	Fundamentação teórica	Leitura de referências bibliográficas e validação das referências;
Etapas 2	Setup Inicial	Criação do repositório remoto e definição da base de dados (CSV). Definição de Pipeline e validação.
Etapas 3	Implementação da Pipeline	Programação do script Python da pipeline; testes na máquina local; extração das métricas de execução nativa (Baseline).

Etapa 4	Avaliação da Abordagem Docker	Resolução de dependências; escrita do Dockerfile; execução dos comandos de <i>build</i> e <i>run</i> ; coleta de métricas de tempo e tamanho.
Etapa 5	Avaliação da Abordagem ReproZip	Instalação e configuração do ReproZip; execução do rastreamento dinâmico (<i>trace</i>); geração do pacote e coleta de métricas correspondentes.
Etapa 6	Redação do Relatório Final	Atualização do README do repositório; Entrega final do trabalho (23/06).

5. Referências Bibliográficas

BOETTIGER, C. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, v. 49, n. 1, p. 71-79, 2015. Disponível em: <https://arxiv.org/abs/1410.0846>. Acesso em: 29 abr. 2026.

CHIRIGATI, F. et al. ReproZip: Computational Reproducibility With Ease. In: *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, p. 2085-2088, 2016. Disponível em: <https://dl.acm.org/doi/10.1145/2882903.2899401>.s Acesso em: 29 abr. 2026.

MALKA, Y. et al. Docker Does Not Guarantee Reproducibility: A Systematic Literature Review and Empirical Study of 5,298 Builds. *ArXiv*, 2026. Disponível em: <https://arxiv.org/abs/2601.12811>. Acesso em: 29 abr. 2026.

DOCKER INC. Best practices for writing Dockerfiles. Disponível em: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/. Acesso em: 29 abr. 2026.

PYTHON SOFTWARE FOUNDATION. Python Packaging User Guide: Managing dependencies. Disponível em: <https://packaging.python.org/en/latest/tutorials/managing-dependencies/>. Acesso em: 29 abr. 2026.

REPROZIP. ReproZip Documentation: Trace, Pack and Unpack. Disponível em: <https://reprozip.org/>. Acesso em: 29 abr. 2026.